

TDT4145 Datamodellering og databasesystemer

10. aug 2024

Sensurveiledning eksamenskont, (oppdatert 16. mai 2025)

Læringsutbyttebeskrivelser for TDT4145

Kunnskaper:

1. Databasesystemer: generelle egenskaper og systemstruktur.
2. Datamodellering med vekt på entity-relationship-modeller.
3. Relasjonsdatabasemodellen for databasesystemer, databaseskjema og dataintegritet.
4. Spørrespråk: Relasjonsalgebra og SQL.
5. Designteori for relasjonsdatabaser.
6. Systemdesign og programmering mot databasesystemer.
7. Datalagring, filorganisering og indeksstrukturer.
8. Utføring av databasespørringer.
9. Transaksjoner, samtidighet og robusthet mot feil.

Ferdigheter:

1. Datamodellering med entity-relationship-modellen.
2. Realisering av relasjonsdatabaser.
3. Databaseorientert programmering: SQL, relasjonsalgebra og database-programmering i Python.
4. Vurdering og forbedring av relasjonsdatabaseskjema med utgangspunkt i normaliseringsteori.
5. Analyse og optimalisering av ytelsen til databasesystemer.

Generell kompetanse:

1. Kjennskap til anvendelser av databasesystemer og forståelse for nytte og begrensninger ved slike systemer.
2. Modellering av og analytisk tilnærming til datatekniske problemer

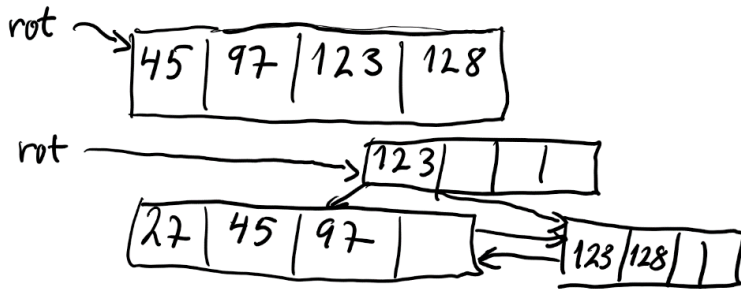
Poenggrenser

Poenggrensene brukt i denne sensuren:

- A 88 poeng
- B 76 poeng
- C 64 poeng
- D 53 poeng
- E 35 poeng

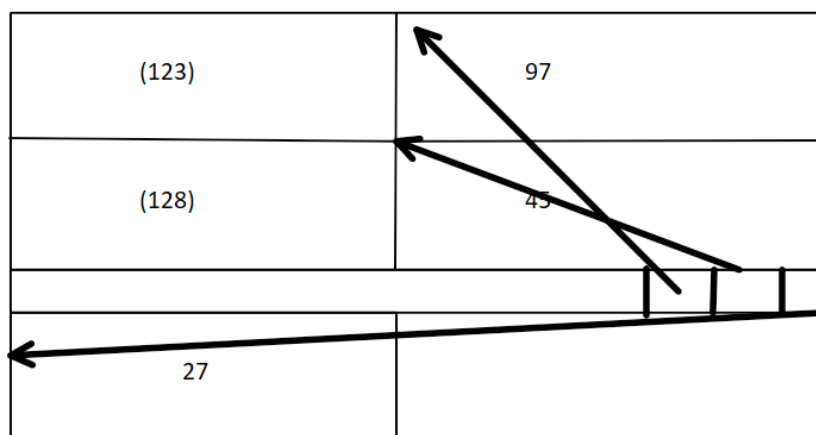
Oppgave 1 (10 %)

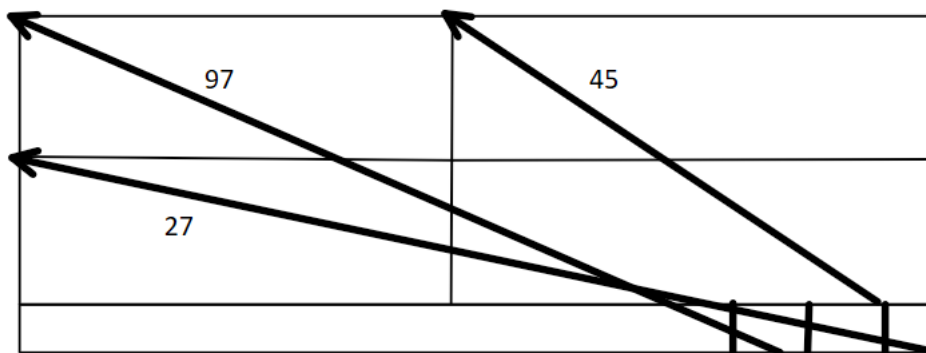
B+-treet tegnet før splitt og etter at alle poster er satt inn



Oppgave 2 (10 %)

Under er først blokka vist uten å ha slettet noen poster ved splittingen. Dermed vil den siste posten få plass utenfor blokka. **Rekkefølgen på postene er viktig. Postene havner i innsettingsrekkefølgen i blokka, men pekervektoren på slutten har postene/nøkklene sorterte.** I den øverste figuren er de to slettede postene antydnet med parenteser rundt nøklene, mens i den siste figuren viser blokka slik den blir etter at den er *kompaktert*, slik at det blir plass til alle postene/nøkklene. Dette må skje før de den siste posten settes inn (27), slik at den får plass. Med i figuren kunne vi tatt med flip/flop, BlockId, pageLSN, prev og next-pekere og evt. andre felter, som f.eks. nRecs (antall poster). Størrelsen på boksene i figuren er omtrentlig slik at 4 * 1000 er 4000 og 96 bytes er igjen til pekervektor og andre blokkfelter. Det holder med 2 bytes til hver peker i pekervektoren.





Oppgave 3 (16 %)

En tilsvarende oppgave som ved ordinær eksamen, men denne gang med delspørsmål med konkrete tall. Her må vi regne med at noen studenter regner annerledes på fyllgrad. Neste år er regnemetode for fyllgrad tatt med i pensum. Noen studenter har antatt bruk av heapfil, men det er feil. Dette er som i MySQL/InnoDB. Postene lagres i B+-trær, og sekundærindekser er også (unclustered) B+-trær. Da er pekerne fra sekundindeksen nøklene brukt i det clustered B+-treet. Når du slår opp i sekundærindeksen, må du da slå opp i primærindeksen etterpå.

- $4096 * 2/3 = 2730,67$. Gir 27 poster per blokk. Da får vi **1482** blokker på level=0.
- 29 byte per post. 25+4 byte (nøkkel til clustered B+-tre). Vi får da 94 poster per blokk. Like mange poster i det unclustered B+-treet som i det clustered. Vi får da **426** blokker på level=0.
- Vi må regne ut høyden på B+-treet (clustered). Hver post level>0 er 12 byte (4+8). Da får vi 227 poster per blokk. 7 blokker på level=1 og 1 blokk på level=2. Da får vi total **3** blokker aksessert for queriet.
- I det unclustered B+-treet er postene 29 byte, mens postene på level>0 er 33 byte (25 + 8 byte). 82 poster per blokk. 6 blokker på level=1. 1 blokk på level=2. Da får vi **6** blokker totalt (3 i unclustered og så 3 i clustered B+-tre).

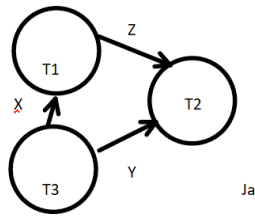
Oppgave 4 (3 %)

- H1: recoverable
- H2: ACA
- H3: Strict

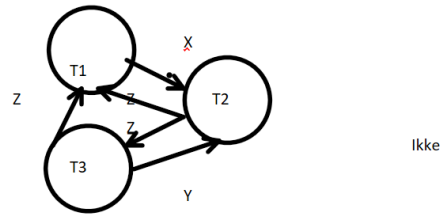
Oppgave 5 (4 %, 2 poeng for hvert rett og -2 for hvert feil svar)

To av historiene er konfliktserialiserbare da presedensgrafene ikke har sykel:

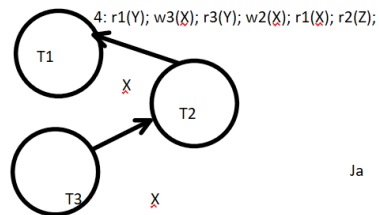
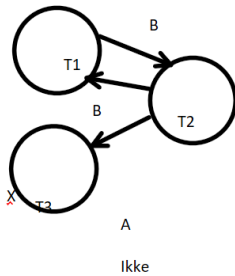
1: $r1(X); r2(Z); r1(Z); r3(X); r3(Y); w1(X); w3(Y); r2(Y); w2(Z); w2(Y)$



2: $r1(X); w2(Z); r1(Z); r3(Z); r3(Y); w2(X); w3(Y); r2(Y); w1(Z); w2(Y);$

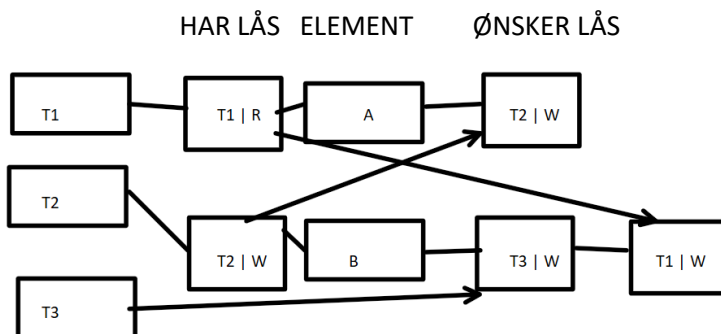


3: $r2(A); r1(B); w2(A); r2(B); r3(A); w1(B); w3(A); w2(B);$



Oppgave 6 (7 %)

Det følgende låsesystemet blir lagd: A og B er elementer som låses. T1, T2, T3 er transaksjoner som har låser. T1 | R betyr leselås for T1. T2 | W betyr skrive-lås for T2.



Oppgave 7 (5 %)

Vi må først beregne DPT etter analysen. Så ser vi på hver loggpost:

102: Må lese inn blokka da $LSN \geq recLSN$

Ikke redo da $PageLSN \geq LSN$

103: Må lese inn blokka da $LSN \geq recLSN$

Ikke redo da $PageLSN \geq LSN$

104: Må lese inn blokka da $LSN \geq recLSN$

Ikke redo da $PageLSN \geq LSN$

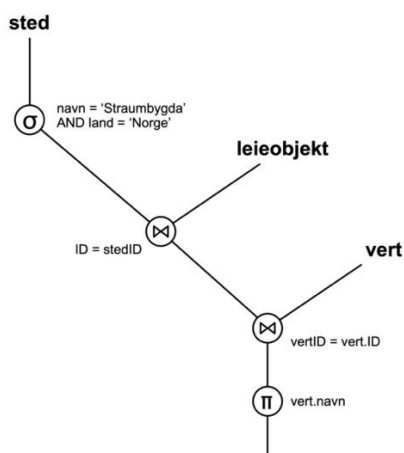
105: Må lese inn blokka da $LSN \geq recLSN$
Ikke redo da $PageLSN \geq LSN$

107: Må lese inn blokka da $LSN \geq recLSN$
Redo da $PageLSN < LSN$

Oppgave 8 (5 %)

Undo må skje etter Redo fordi databasen må være i en konsistent tilstand når undo skal skje, på den måten at endringen som skal undoes er allerede gjort. I det forrige eksemplet ville undo før redo, gjort at T2 ville ha undoet sin loggpost med $LSN=107$ først. Da ville ikke databasen ha riktig tilstand for element D til å gjøre undo da 107 ikke ennå er redoet.

Oppgave 9 (5 %)



Oppgave 10 (10 %)

```
select leieobjekt.ID, leieobjekt.navn, avg(antallStjerner) AS
'gjennomsnitt'
from sted inner join leieobjekt on (ID=stedID) inner join anmeldelse
on (leieobjekt.ID = objektID)
where sted.navn = 'Utskarpen' and sted.land = 'Norge'
group by leieobjekt.ID, leieobjekt.navn
order by gjennomsnitt DESC
```

Det er ikke nødvendig å ha med `leieobjekt.navn` i «group by»-delen, og det er ikke nødvendig å lage aliaset «gjennomsnitt».

Oppgave 11 (10 %)

resultat(UtøverID, Arena, Dato, Øvelse, Resultat) – **BCNF** fordi «venstresiden» i alle ikke-trivielle funksjonelle avhengigheter er en supernøkkel.

utøver(UtøverID, Navn, Alder) – **BCNF** fordi «venstresiden» i alle ikke-trivielle funksjonelle avhengigheter er en supernøkkel.

arenaer(Arena, Sted) – **BCNF** fordi «venstresiden» i alle ikke-trivielle funksjonelle avhengigheter er en supernøkkel.

Dekomponeringen har **attributtbevaring** fordi alle attributter i friidrettResultat er med i minst en (komponent-)tabell.

Dekomponeringen har **bevaring av funksjonelle avhengigheter** fordi alle funksjonelle avhengigheter i friidrettResultat er representert i minst en (komponent-)tabell:

- Arena -> Sted i arenaer-tabellen
- UtøverID -> Navn, Alder i utøver-tabellen
- UtøverID, Arena, Dato, Øvelse -> Resultat i resultat-tabellen

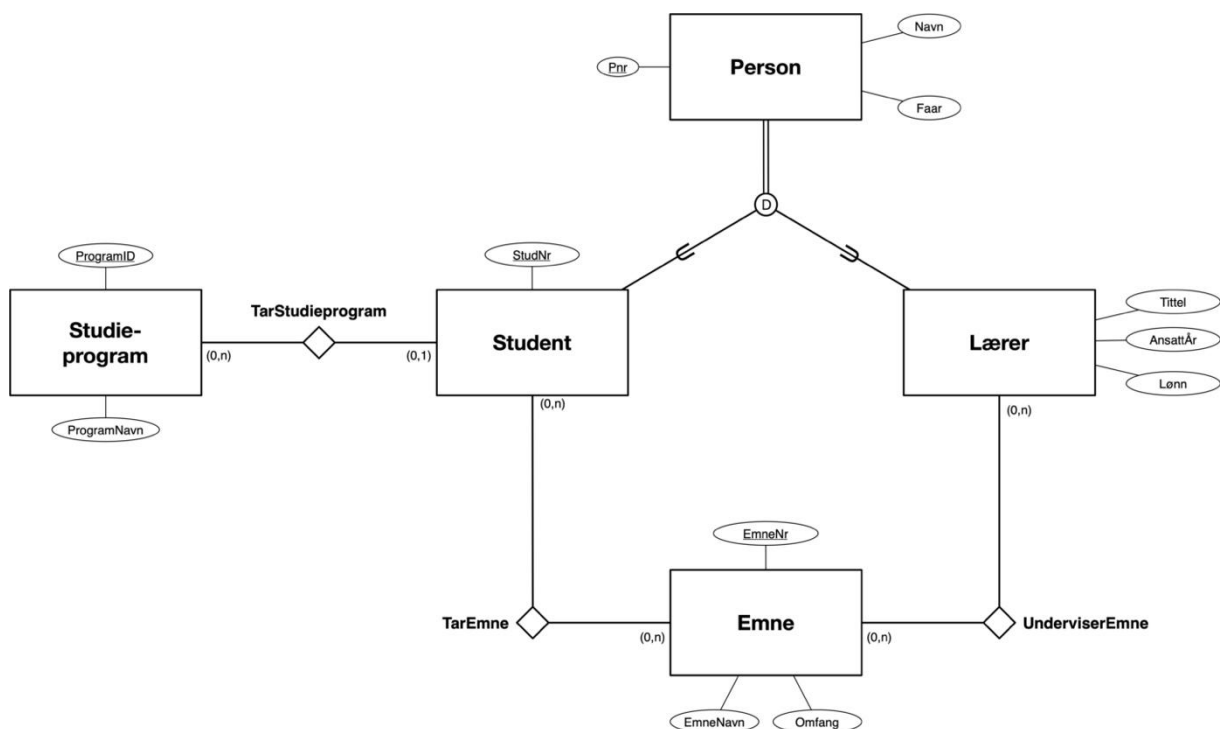
Dekomponeringen har tapsløst-join-egenskapen fordi resultat-tabellen og utøver-tabellen joiner tapsløst (felles attributt utøverID er supernøkkel i utøver-tabellen). Dette join-resultatet joiner så tapsløst med arenaer-tabellen (felles attributt Arena er supernøkkel i arena-tabellen).

Tapsløst-join-egenskapen kan også vises ved å bruke «tabell-metoden».

Oppgave 12 (5 %)

(xt-1, 23 mm, ef-42), (xt-1, 50 mm, ef-42), (xt-5, 23 mm, ef-42), (xt-5, 23 mm, ef-60), (xt-5, 35 mm, ef-42) og (xt-5, 35 mm, ef-60).

Oppgave 13 (10 %)



I denne løsningen har vi lagt til grunn følgende antagelser, ut over det som går frem av relasjonsmodellen:

- Alle personer er enten student eller lærer
- En person kan ikke være både student og lærer
- Studieprogram kan eksistere uten studenter som tar programmet, og programmet kan ha mange studenter
- En lærer trenger ikke å undervise emner
- En student trenger ikke å ta emner

- Et emne trenger ikke å ha studenter og trenger heller ikke å ha noen lærer som underviser emnet

Det er greit å gjøre andre forutsetninger så lenge de er i overenstemmelse med relasjonsmodellen, og det er samsvar mellom forutsetninger og foreslått ER-modell.